

ESP32 Wi-Fi Drive & Weapon System

ESP32-S3 · DRV8833 Dual Motor Driver · Browser Joystick — No RC Transmitter/Receiver

This guide covers the fully microcontroller-based control architecture: an ESP32-S3 board runs onboard firmware that creates its own Wi-Fi network, serves a joystick web page to your phone or laptop, and drives two N20 gear motors through a DRV8833 dual H-bridge plus a brushless weapon ESC — all direct GPIO, no RC transmitter, receiver, or hobby radio gear anywhere in the loop. It combines a parts list, pinout, a custom wiring diagram, a minimal-footprint harness build guide, firmware configuration, a pre-flight checklist, and troubleshooting into one document.

1 · System Overview	2 · Parts List	3 · Arduino IDE & Upload	4 · Pin Reference
5 · Wiring Diagram	6 · Harness Build Guide	7 · Firmware Setup	8 · Boot & ESC Arming
9 · Driving the Bot	10 · Pre-Flight Checklist	11 · Troubleshooting	12 · Safety Notes

1 · System Overview

An ESP32-S3 board is the whole brain of the robot. On boot, it creates its own Wi-Fi access point — the bot itself is the network, nothing else is needed to join it — and hosts a single web page containing a virtual joystick and a weapon throttle slider. Your phone or laptop connects to that network and opens a browser to drive. The board translates joystick input into PWM signals for a DRV8833 dual-motor driver (running the two N20 drive motors) and a microsecond pulse for the brushless weapon ESC, exactly the same pulse format a hobby servo or RC receiver would send — the ESP32 is just generating it directly instead of decoding it from a radio.

BRAIN

ESP32-S3 DevKit

Wi-Fi AP + web server, PWM motor control, RGB status LED, software failsafe.

DRIVE

DRV8833 + 2× N20

Dual H-bridge, 20kHz PWM, brakes to a stop instead of coasting.

WEAPON

Slim 15A 2S Brushless ESC

Throttle-only, no reverse. 1000–2000µs pulse, 1000µs = off.

The firmware boots in a hard-latched **STOP** state every time — nothing moves until the driver taps **ACTIVATE** in the browser. If the phone loses the Wi-Fi link, closes the tab, or simply stops sending frames for more than 1.2 seconds, the bot re-latches itself automatically. A single **STOP ALL** button on screen, the **Escape/Space** keys on a laptop, and even backgrounding the browser tab all trigger the same emergency-stop path.

Caution — The weapon ESC is throttle-only, exactly like the RC-based build: 1000µs is its true "off," there's no centered neutral. The firmware maps the browser's 0–180 weapon value directly onto that 1000–2000µs range, but the same "unarmed = idle" gate that makes **STOP** mean **STOP** is only as good as the wiring behind it — double-check §6 and §11 before powering the weapon.

2 · Parts List

PART	SPEC	QTY
2S LiPo battery	7.4V nominal, high discharge rating	1
ESP32-S3 DevKit	Any board with an onboard addressable RGB LED (GPIO21) is a drop-in match	1
DRV8833 dual motor driver	Breakout board, 2-channel H-bridge	1
N20 gear motor	~500 RPM class @ 6V nominal	2
Brushless weapon ESC	Slim profile, 15A, 2S, throttle-only	1
Weapon motor	Brushless, sized to the ESC and your weapon	1
JST-XH 3-pin connector pairs	Keyed signal/logic connectors (see §6)	2–3
XT30 (or similar) power connector	Battery-side power connector, polarized	1 pair
22–26 AWG silicone wire, heat-shrink	Signal/logic runs; 18–20 AWG for battery/motor runs	as needed

No hobby transmitter, receiver, or servo tester is used anywhere in this build — every signal in Section 4 originates from the ESP32's own GPIO pins.

3 · Arduino IDE Setup & Uploading the Sketch

This is the software side: getting a free program called the Arduino IDE onto your computer, teaching it about the ESP32-S3-Zero board, and using it to send ("upload" or "flash") the BattleBots.ino sketch onto the board over a USB cable. None of this needs any programming knowledge — you're following a checklist, not writing code.

3.0 Get the sketch onto your computer first

The sketch lives in a GitHub repository — you do not need a GitHub account to grab a copy, and you don't need to already have the file to follow this guide. Pick whichever of these is easiest:

- **Easiest — direct download (no software needed):** open github.com/thefuzzman/WSP_YM_BB in a browser, click the green **Code** button, then **Download ZIP**. Once it downloads, double-click the .zip file (Windows/macOS both unzip automatically, or right-click → Extract on Windows) to get a normal folder you can browse.

- **Clone with Git (if you already use Git):** open a terminal and run `git clone https://github.com/thefuzzman/WSP_YM_BB.git` — this downloads the repository into a new WSP_YM_BB folder in your current directory.

Either way, you end up with a normal folder on your computer. Locate the `BattleBots.ino` sketch file inside it — that's the file you'll open in §3.5. Keep the whole folder together; the `.ino` file expects to sit in a folder of the same name if any supporting files are added later.

3.1 Install the Arduino IDE 2

The Arduino IDE is the free program you'll use for everything in this section. Download it from arduino.cc/en/software — pick the installer for your computer:

- **Windows:** download the "Windows Win 10 and newer" installer (.exe). Double-click it once downloaded, click through the installer (default options are fine), and allow any "install USB driver" prompt that appears partway through — that driver is what lets Windows talk to the board over USB.
- **macOS:** download the macOS installer (.dmg, choose Apple Silicon or Intel to match your Mac — Apple menu → About This Mac tells you which). Open the .dmg and drag the Arduino IDE icon into the Applications folder. The first time you open it, macOS may warn the app was downloaded from the internet — click "Open" to proceed.
- **Linux:** download the AppImage. Make it executable (right-click → Properties → Permissions → "Allow executing file as program", or run `chmod +x` on it from a terminal), then double-click to run it. No further install step is needed.

Launch the IDE once after installing just to confirm it opens — you should see a mostly-blank code window with a horizontal toolbar across the top.

3.2 Add the ESP32 board support

The Arduino IDE doesn't know about ESP32 boards out of the box — you have to point it at Espressif's (the chip maker's) board definitions first, once, ever, on each computer you use.

1. Open **Preferences**: on Windows/Linux this is File → Preferences; on macOS it's Arduino IDE → Settings (or Preferences).
2. Find the field labeled **Additional boards manager URLs** near the bottom of the Settings tab, and click the small icon at the right edge of that field to open a bigger text box.
3. Paste in exactly this address: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
4. Click OK to close that box, then OK again to close Preferences.
5. Open the **Boards Manager** — the icon that looks like a stack of boards/squares on the left-hand toolbar (or Tools → Board → Boards Manager).

6. Type **esp32** into the search box at the top. Look for the entry titled **esp32 by Espressif Systems** and click **Install**. This downloads for a minute or two — let it finish completely before moving on.

3.3 Install the ESP32Servo library

A "library" is a small add-on that teaches the IDE extra tricks — in this case, generating the precise servo/ESC-style pulses the sketch needs.

1. Open the **Library Manager** — the icon that looks like an open book on the left-hand toolbar (or Tools → Manage Libraries).
2. Type **ESP32Servo** into the search box. Find the entry simply named **ESP32Servo** by Kevin Harrington/John K. Bennett, and click **Install**.
3. If a popup asks about installing dependencies, click **Install All**.

3.4 Connect the ESP32-S3-Zero and select it in the IDE

1. Plug the ESP32-S3-Zero into your computer with a USB-C cable that supports data (not a charge-only cable — if in doubt, use the cable that came with the board). Plug into the port labeled **USB** on the board itself if it has two ports.
2. In the top toolbar, click the **board selector dropdown** (it may say "Select Board" the first time) and choose **Select other board and port...**
3. In the search box, type **ESP32S3 Dev Module** and select it from the list — this generic entry is the correct one for the ESP32-S3-Zero.
4. In the same window, look under Ports for something like `/dev/cu.usbmodemXXXX` (Mac), COM3 (Windows), or `/dev/ttyACM0` (Linux) and select it — this is your board's USB connection. If no port appears, unplug and re-plug the USB cable, or try a different cable/port.
5. Click **OK**. The toolbar should now show "ESP32S3 Dev Module" and a port name.

If nothing shows up: some ESP32-S3-Zero boards need to be put into "download/boot" mode manually the first time. Hold the button labeled **BOOT** on the board, briefly tap **RESET** while still holding BOOT, then release BOOT — the board is now ready to accept an upload. You only need to do this if the automatic upload in §3.6 fails.

3.5 Open the sketch and check the editable fields

In the IDE, use File → Open... and select `BattleBots.ino`. A handful of settings near the top of the file are meant to be changed by you — everything else can be left alone:

- **Connection mode** — exactly one of `#define CONNECTION_WIFI` / `#define CONNECTION_BLE` active (not commented with `//`).
- **Weapon type** — exactly one of `#define WEAPON_SERVO` / `#define WEAPON_ESC` active. This

build uses WEAPON_ESC.

- **Pin numbers** — WEAPON_PIN, L_IN1/L_IN2, R_IN1/R_IN2 should match Section 4's table (GPIO 6, 7, 8, 9, 10) unless your wiring differs.
- **Wi-Fi name/password** — apPassword defaults to battle123; change it if you want a private network name/password at a shared event.

Don't touch anything outside these labeled blocks unless a later section of this guide specifically tells you to — the rest of the sketch is the logic that makes the safety behavior (stop-latch, failsafe, ESC arming) work correctly.

3.6 Compile and upload

1. Click the **checkmark (✓)** icon in the top-left of the toolbar — this is **Verify/Compile**. It checks the sketch for errors without touching the board yet. Wait for the black console at the bottom to say "Done compiling" — this can take 30–90 seconds the first time.
2. If it reports an error instead, re-read §3.5 for a missing/duplicate #define, or re-check §3.2/3.3 — a red error mentioning "Servo.h not found" means the ESP32Servo library didn't install correctly.
3. Once compiling succeeds, click the **right-arrow (→)** icon right next to it — this is **Upload**. The IDE compiles again, then sends the program to the board over USB. You'll see the console scroll with progress and percentage numbers, and the board's onboard LED may flicker during this step.
4. When you see "**Hard resetting via RTS pin...**" or "**Done uploading**" in the console, the upload succeeded. The board will automatically restart and begin running the new sketch.
5. Open **Tools** → **Serial Monitor** and set the baud rate in its bottom-right dropdown to **115200** — you should see startup text appear (Wi-Fi network name, boot sequence messages) confirming the sketch is running. This window is also how you'll use the BENCH safety-test command mentioned in §12.

Every time you change a setting in §3.5, you must re-upload (click → again) for the change to take effect — editing the file alone does nothing until it's re-flashed to the board.

4 · Pin Reference (ESP32-S3)

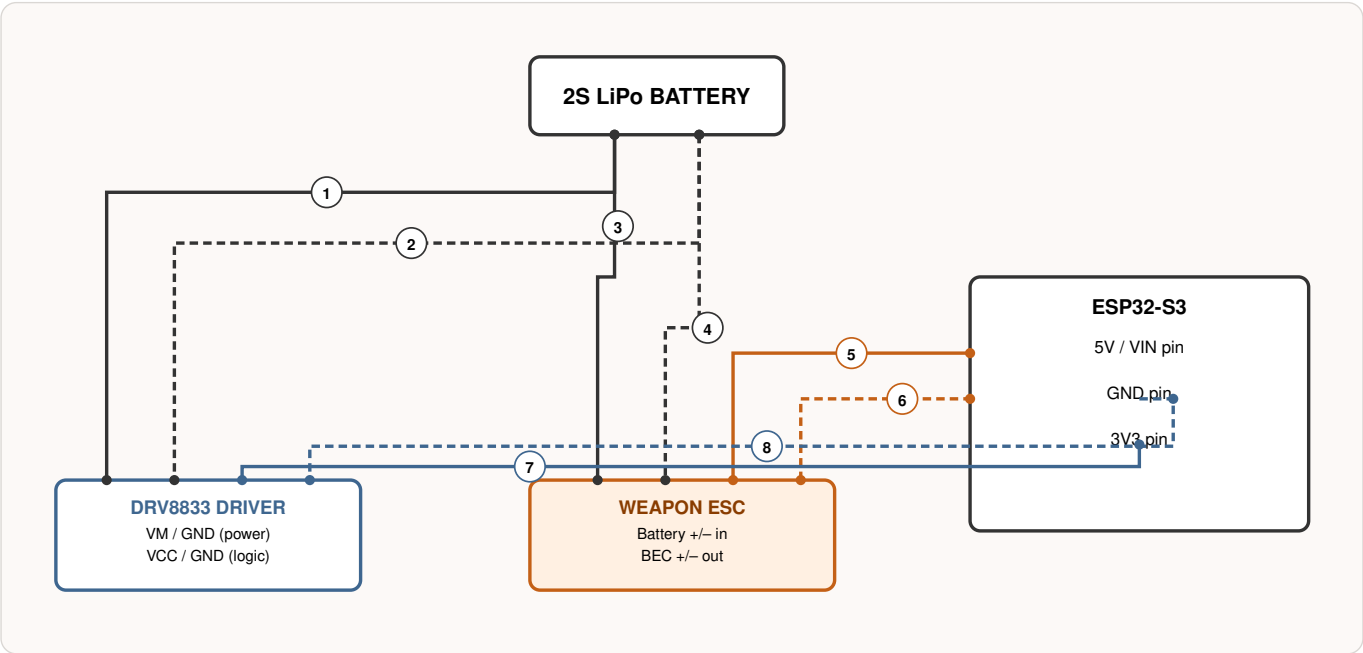
SIGNAL	GPIO	GOES TO
WEAPON_PIN	GPIO 6	Weapon ESC signal wire
L_IN1	GPIO 7	DRV8833 AIN1 (left motor)
L_IN2	GPIO 8	DRV8833 AIN2 (left motor)
R_IN2	GPIO 9	DRV8833 BIN2 (right motor)
R_IN1	GPIO 10	DRV8833 BIN1 (right motor)
RGB_LED (built-in)	GPIO 21	Onboard NeoPixel — no external wiring

These pin numbers are the firmware's EDIT AS NEEDED block — if your board's silkscreen uses different labels, or a motor spins the wrong way, change the numbers in the sketch (swap L_IN1/L_IN2 to reverse a motor) rather than re-wiring the harness.

5 · Wiring Diagram

Split into three schematics by function — power, logic/signal, and motor outputs — so every individual wire is drawn as its own line from its exact source pad to its exact destination pad. The pad-to-pad table at the end is the authoritative reference if a label is ever ambiguous.

4.1 Power wiring (8 wires)

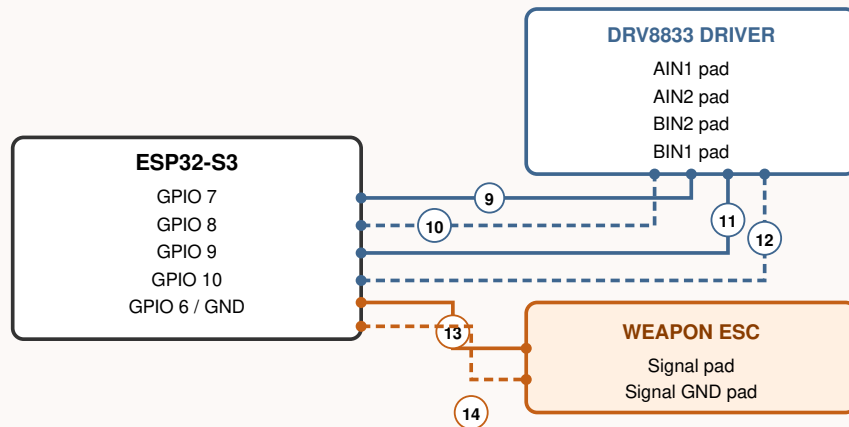


1 Batt+ → DRV8833 VM

2 Batt- → DRV8833 GND

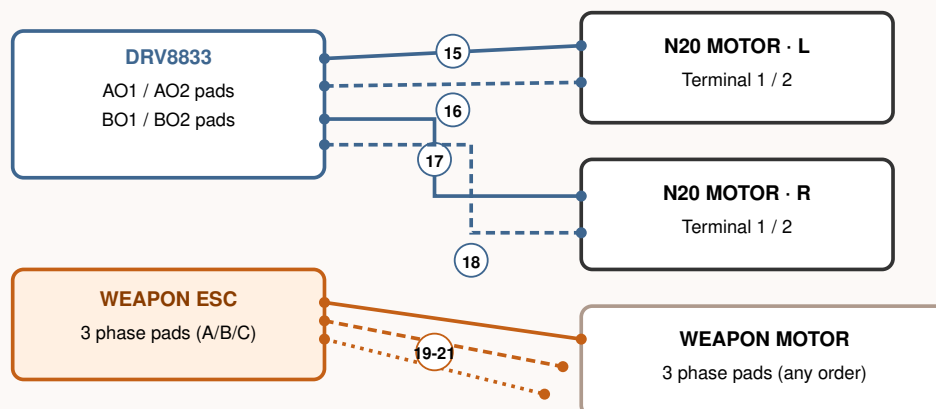
- 3 Batt+ → ESC batt+ pad
- 5 ESC BEC red → ESP32 5V/VIN
- 7 ESP32 3V3 → DRV8833 VCC
- 4 Batt- → ESC batt- pad
- 6 ESC BEC black → ESP32 GND
- 8 ESP32 GND → DRV8833 GND (logic)

4.2 Logic & signal wiring (6 wires)



- 9 GPIO7 → DRV8833 AIN1
- 11 GPIO9 → DRV8833 BIN2
- 13 GPIO6 → ESC signal pad
- 10 GPIO8 → DRV8833 AIN2
- 12 GPIO10 → DRV8833 BIN1
- 14 ESP32 GND → ESC signal GND pad

4.3 Motor output wiring (7 wires)



- 15 DRV8833 AO1 → N20(L) term. 1
- 17 DRV8833 BO1 → N20(R) term. 1
- 16 DRV8833 AO2 → N20(L) term. 2
- 18 DRV8833 BO2 → N20(R) term. 2

19–21 ESC 3 phase pads → weapon motor 3 phase pads (any order)

RGB status LED is built onto the ESP32-S3 board at GPIO 21 — no external wire needed. If a motor spins backward, swap wires 15↔16 or 17↔18. If the weapon spins backward, swap any two of wires 19–21.

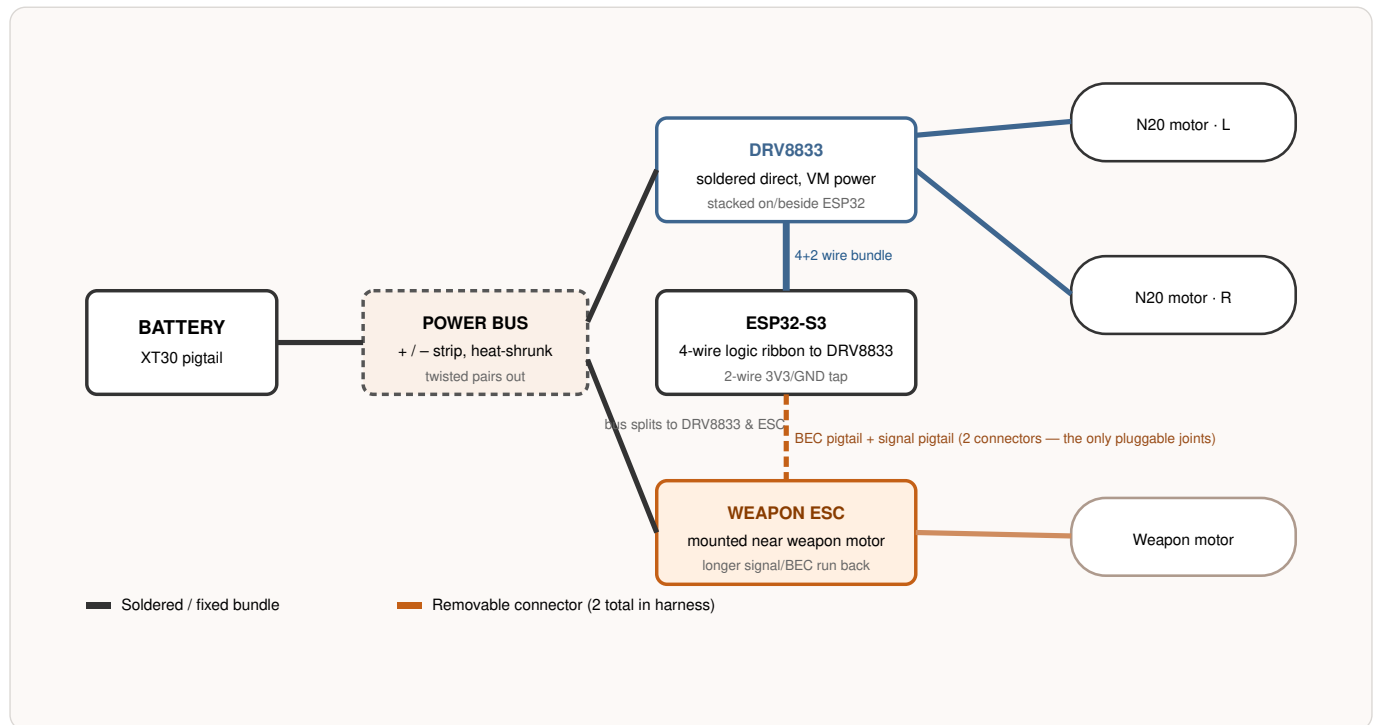
WIRE	FROM	TO
1	Battery +	DRV8833 VM pad
2	Battery -	DRV8833 GND pad
3	Battery +	Weapon ESC battery + pad
4	Battery -	Weapon ESC battery - pad
5	Weapon ESC BEC red	ESP32 5V / VIN pin
6	Weapon ESC BEC black	ESP32 GND pin
7	ESP32 3V3 pin	DRV8833 VCC pad
8	ESP32 GND pin	DRV8833 GND pad (logic-side)
9	ESP32 GPIO 7	DRV8833 AIN1 pad
10	ESP32 GPIO 8	DRV8833 AIN2 pad
11	ESP32 GPIO 9	DRV8833 BIN2 pad
12	ESP32 GPIO 10	DRV8833 BIN1 pad
13	ESP32 GPIO 6	Weapon ESC signal pad
14	ESP32 GND pin	Weapon ESC signal ground pad
15	DRV8833 A01 pad	N20 (L) terminal 1
16	DRV8833 A02 pad	N20 (L) terminal 2
17	DRV8833 B01 pad	N20 (R) terminal 1
18	DRV8833 B02 pad	N20 (R) terminal 2
19–21	Weapon ESC 3 phase pads	Weapon motor 3 phase pads (any order)

RGB status LED is built onto the ESP32-S3 board at GPIO 21 — no external wire needed. If a motor spins backward, swap that motor's two wires (15↔16 or 17↔18) rather than re-signing anything in firmware. If the weapon spins backward, swap any two of wires 19–21.

6 · Minimal-Footprint Harness Build Guide

The goal here is the smallest, lightest bundle of wiring that still lets you pull the electronics stack for repairs without re-soldering anything. Combat robots are weight- and space-constrained, so this harness is built around three ideas: **one shared power bus**, **short direct signal runs**, and **keyed connectors so nothing can be plugged in backward or wrong**.

Physical harness layout (as built, not schematic)



This is the physical bundle layout, not the electrical schematic in §5 — it shows how the wires are grouped and routed on the chassis. DRV8833 sits stacked against the ESP32 so their 6-wire logic bundle stays under ~3cm; the weapon ESC sits near the weapon motor, so only its BEC and signal pigtails need to travel back to the ESP32 — the two removable connectors in the whole build.

5.1 Plan the physical layout first

Before cutting any wire, dry-fit the four components — battery, DRV8833, ESP32, weapon ESC — in their final positions in the shell. Stack the DRV8833 directly on or beside the ESP32 if possible; this is the single biggest footprint saver, since it turns four logic wires into a ~1cm hop instead of a routed cable. Place the weapon ESC close to the weapon motor (its phase wires are the ones you don't want to extend) and let its signal/BEC leads be the longer run back to the ESP32 instead.

5.2 Build the shared power bus

1. Solder the battery's XT30 pigtail to two short buses (a "+" strip and a "-" strip) rather than daisy-chaining connector to connector — a small 2-pad perfboard strip or twisted power-only pigtail both work, and both keep the joint count low.
2. From the bus, run one + and one – lead to the DRV8833's VM/GND motor-power input, and a second + / – pair to the weapon ESC's battery input. Keep both runs as short as the layout allows — these carry the highest current in the build.
3. Twist each power pair (+ with its matching –) along its whole length. Twisting cancels out the small radiated noise a switching PWM signal creates, which otherwise can couple into the nearby low-current logic wires and cause flaky behavior.
4. Heat-shrink every solder joint on the bus individually, then group the whole bus under one larger piece of heat-shrink so it reads as a single cable, not a rat's nest.

5.3 The single BEC power hop

Only one 5V source exists in this build: the weapon ESC's BEC. Run its red (+5V) and black (GND) BEC leads directly to the ESP32's 5V/VIN and GND pins — a two-conductor JST-XH pigtail is a clean way to make this removable without a bare-wire splice. Do not also connect any 5V/BEC output from anywhere else; there's only the one here, so there's no conflict to manage, unlike the RC build's three-ESC version.

5.4 Signal & logic harness (the small-footprint part)

1. Use 4 short 26 AWG silicone leads for GPIO 7/8/9/10 → DRV8833 AIN1/AIN2/BIN2/BIN1. If the two boards are stacked or mounted side-by-side, these can be under 3cm each — solder them directly, no connector needed, since this joint is never meant to be unplugged in the field.
2. Bundle those 4 wires together with a single small piece of heat-shrink or a spot of hot glue every couple of centimeters so they move as one ribbon instead of four loose strands that can snag.
3. Tie the DRV8833's logic VCC to the ESP32's 3.3V pin, and its GND to the shared ground — this is a 2-wire addition to the same small bundle.
4. For the weapon signal (GPIO6 → ESC signal pad), run one 26 AWG lead plus a shared ground back to the ESC. If the ESC is mounted far from the ESP32 (recommended — keep it near the weapon motor per §5.1), terminate this one wire in a single-pin or 2-pin JST-SH connector at the ESP32 end so the harness un-plugs as one unit during teardown.
5. Every ground in the system — battery bus, DRV8833 logic GND, ESP32 GND, weapon ESC GND — must ultimately tie back to one common point (a "star ground"). Don't let grounds daisy-chain through multiple boards; wire each one back to the main power bus's – terminal directly.

5.5 Connector strategy

Use exactly two connector families in this build, chosen so nothing can be cross-plugged by accident:

- **Power (battery, motor supply, BEC hop):** XT30 or a similarly-rated 2-pin polarized connector, sized well above your expected current draw.
- **Signal/logic (weapon signal pigtail only):** JST-SH or JST-XH, small pitch, low profile — these are the connectors already sized for low-current signal wire and won't add bulk.

Everything else (the 4-wire motor-driver logic bundle, the 3.3V/GND taps) is short enough to solder directly rather than adding a connector's weight and failure point for a joint you won't be unplugging often.

5.6 Final routing & strain relief

- Route all wiring away from the weapon's swept area and any moving drivetrain parts — a snagged wire during weapon spin-up is a common cause of build damage.
- Leave a small service loop (a bit of slack, not tension) at every connector so vibration loads the strain-relief boot, not the solder joint.
- Zip-tie or hot-glue the finished harness to the chassis at 2–3 points so nothing can shift under impact.
- Once assembled, this whole harness should be one bundle with exactly two unpluggable joints (battery connector, weapon-ESC signal pigtail) — everything else is fixed, which is what keeps the footprint minimal and the build reliable.

7 · Firmware Configuration

The sketch is organized around a handful of `EDIT AS NEEDED` blocks near the top of the file — everything you're expected to customize lives there, not scattered through the code.

1. **Connection mode:** exactly one of `#define CONNECTION_WIFI` or `#define CONNECTION_BLE` should be un-commented. Wi-Fi is the simplest to start with — no companion app needed, just a browser. Comment out the one you're not using by putting `//` in front of it.
2. **Weapon type:** similarly, exactly one of `#define WEAPON_SERVO` or `#define WEAPON_ESC` should be active. For the brushless weapon ESC in this guide, that's `WEAPON_ESC`.
3. **Pin numbers:** confirm `WEAPON_PIN`, `L_IN1/L_IN2`, and `R_IN1/R_IN2` match Section 4's table (or your board's actual wiring, if it differs). If a wheel spins backward once powered up, swap that motor's two IN pin numbers in the sketch and re-upload — much easier than re-soldering.
4. **Wi-Fi network name/password:** the SSID is auto-generated per-board from its chip ID (AntBot-S3- . . .) so two bots never collide; the password defaults to `battle123` — change `apPassword` if

you don't want other builders at an event joining your bot's network.

5. Upload the sketch to the ESP32-S3 over USB using the Arduino IDE, with the ESP32Servo library installed beforehand (Library Manager → search "ESP32Servo").

8 · Boot Sequence & ESC Arming

Every power-up follows the same fixed sequence, and it's worth understanding because it explains what the status LED is telling you:

1. Motor driver pins are configured and both N20 motors are immediately braked — nothing spins on power-up no matter what.
2. The weapon ESC signal pin is attached and immediately held at 1000µs (zero throttle) continuously for 3 seconds while the ESC runs through its own power-on arming tones. **Keep hands and objects clear of the weapon during this window** — some ESCs twitch briefly while arming.
3. The bot boots **stop-latched**: the firmware will not act on any drive or weapon command, from any source, until the browser's ACTIVATE button is pressed.
4. The Wi-Fi access point comes up and the status LED turns solid cyan — the network is ready, waiting for a browser to connect.
5. Once a browser loads the page and taps ACTIVATE, the bot arms at zero throttle/neutral drive (never at whatever the slider last showed) and the LED turns green.

9 · Driving the Bot

1. On your phone or laptop, open Wi-Fi settings and join the network named AntBot - S3 - ... using password battle123 (or your changed password).
2. Open a browser and go to 192 . 168 . 4 . 1. The joystick page loads automatically — no app install required.
3. Tap the green **ACTIVATE BOT** button. The page switches to the driving screen and the bot's LED turns green.
4. Drag inside the left circular pad to drive: forward/back and left/right blend together into tank-style differential steering, same as pushing a single stick.
5. Use the weapon slider on the right (or the 25/50/75/100% preset buttons) to set weapon speed; tap **WEAPON OFF** to cut it instantly while staying armed and drivable.
6. Tap the red **STOP ALL** button (or hit Space/Escape from a laptop) any time to hard-stop and re-

latch — you'll need to press **ACTIVATE** again to resume.

A small dot in the corner of the screen flashes green on every successful command frame — if it turns red or stops flashing, the link to the bot has degraded; treat that as a cue to stop driving and check the connection.

10 · Pre-Flight Checklist

Run through this every time before connecting the main battery, and especially after any wiring or firmware change:

- ☐ Battery voltage checked — do not run a 2S pack below its safe discharge cutoff.
- ☐ Visual check: no exposed/shorted wiring, all heat-shrink intact, no loose solder joints at the DRV8833 logic bundle or weapon ESC pads.
- ☐ Weapon area clear of fingers, tools, and loose objects before applying power (ESC arming tones will run automatically).
- ☐ Confirm the status LED goes solid cyan (Wi-Fi ready) within a few seconds of power-up — if it doesn't, do not proceed to **ACTIVATE**.
- ☐ Connect your phone/laptop to the bot's Wi-Fi network and load 192.168.4.1 before **ACTIVATE**.
- ☐ With the bot lifted off the ground (wheels free-spinning), **ACTIVATE** and test drive at low joystick deflection before doing anything else.
- ☐ Briefly test the weapon at low throttle and confirm the **WEAPON OFF** button and slider-to-zero both stop it fully.
- ☐ Close the browser tab (or walk out of Wi-Fi range) while the bot is armed and confirm the failsafe stops it within ~1.2 seconds, rather than freezing it at speed.
- ☐ Test **STOP ALL** and confirm the bot re-latches and requires **ACTIVATE** again to resume.

11 · Troubleshooting Reference

SYMPTOM	LIKELY CAUSE	FIX
Nothing powers on / no Wi-Fi network appears	BEC hop from weapon ESC to ESP32 not connected, or battery fault	Check battery connections first, then confirm the ESC's BEC leads reach the ESP32's 5V/GND pins
Browser can't reach 192.168.4.1	Phone still on cellular data, or not joined to the bot's SSID	Confirm you're joined to AntBot-S3-... and disable "assisted" auto-switch networks on the phone
A drive wheel spins the wrong direction	Motor wire polarity or IN pin swap	Swap that motor's two wires at the DRV8833 output, or swap its two IN pin numbers in the sketch
Robot turns opposite of joystick input	Left/right motor outputs swapped at the DRV8833	Swap the two motors' connections at the driver board
Weapon won't spin, even at full slider	Bot not activated, or ESC still mid-arming	Confirm LED is green (armed), and that 3 seconds passed since power-up for ESC arming
Weapon spins the wrong direction	Phase order on the brushless motor	Swap any two of the three motor phase wires at the ESC
Bot disarms itself constantly while driving	Weak Wi-Fi signal or too many stacked/queued frames tripping the 1.2s failsafe	Move closer to the bot; check the on-screen ping dot for dropped frames
Vertical weapon slider is hard to use on some phones	Browser doesn't honor vertical range-input styling	Use the 25/50/75/100% preset buttons instead of dragging the slider
Serial monitor shows garbage text	Wrong baud rate	Set the Serial Monitor to 115200 baud

12 · Safety Notes

- The weapon ESC is throttle-only with no reverse — "off" only happens at the very bottom of its signal range (1000μs). Verify this on every power-up before trusting it.
- Always disconnect the main battery when the bot isn't actively being driven or tested — Wi-Fi range and a software failsafe are not a substitute for physically removing power.
- Keep hands, tools, and loose clothing clear of the weapon any time the battery is connected, even before ACTIVATE — the ESC arming sequence briefly drives the weapon signal line during boot.
- Use the Serial BENCH command (typed into the Serial Monitor from §3.6) only with the weapon blade/drum physically removed — it deliberately ramps the weapon to 25% without needing the browser at all, specifically so you can isolate hardware problems from UI problems.
- Use eye and body protection during weapon testing, and test with the robot restrained or lifted clear of the ground for the first run after any wiring or firmware change.
- Re-run the full Pre-Flight Checklist after any rewiring or re-flash, not just before a match.

Build guide compiled for the ESP32-S3/DRV8833/Wi-Fi-joystick combat robot configuration. Cross-check part-specific details (exact ESC arming behavior, board pin labeling) against your actual components, since manufacturers vary.